

1. Módulo 1 – Linguagens de Programação

Linguagens

- A **linguagem de programação** pode ser definida como o meio formal de comunicação com o computador. Cada linguagem é composta por um conjunto formal de regras semânticas e sintáticas.
- **Algoritmo** - Sequência de instruções ou regras bem definidas para realizar uma tarefa. A linguagem de programação estrutura o algoritmo para o computador

Níveis de Linguagem

- **Linguagem de máquina** → Linguagem compreendida pelo computador (instruções sequenciais escritas em bits)
- **Linguagem Assembly** → Utiliza abreviações textuais para representar diretamente a linguagem de máquina
- **Linguagens de alto nível** → A programação se aproxima da linguagem natural humana. As instruções são convertidas em linguagem de máquina pelos compiladores ou interpretadores.

Compilador e Interpretador

- **Compilador** → Traduz todo o código fonte de um determinado programa para a linguagem de máquina, antes da execução.
- **Interpretador** → Lê, traduz e executa cada uma das linhas do código, em tempo de execução do programa.

Paradigmas de Programação

- **Paradigma de Programação** → Refere-se as formas de pensar e organizar a escrita de programas.
- **Paradigma Imperativo** → Estilo de programação na qual o programador descreve passo-a-passo o que deve ser feito pela máquina. Exemplo: Assembly.
- **Paradigma procedural** → Estilo de programação baseado na chamada de funções ou procedimentos (A modularização do Imperativo). Exemplo: C.
- **Paradigma orientado à objetos** → Estilo de programação que organiza a execução em torno do conceito de objetos, aproximando da noção real de objetos no nosso mundo. Exemplo: Java.

- **Paradigma Declarativo** → Estilo de programação na qual se descreve o resultado esperado, não o passo-a-passo a ser executado. Exemplo: SQL (Não é linguagem de programação).
- **Paradigma lógico** → Estilo de programação declarativo, que se baseia no conceito da lógica proposicional. Exemplo: Prolog.
- **Paradigma funcional** → Estilo de programação baseado em funções matemáticas, evitando dados mutáveis e estados compartilhados. Exemplo: Lisp.

2. Modulo 2: Linguagem C

- **Pré-processamento da compilação** → Expande as diretivas (#include)
- **Compilação em C** → Traduz o código expandido para linguagem *assembly*.
- **Montagem** → Converte o *assembly* em código objeto binário (.o ou .obj).
- **Linkagem em C** → Resolve as referências do código objeto às bibliotecas externas necessárias, gerando o executável final.
- **Constantes e Variáveis** → Consistem em espaços na memória do computador que ficam reservado para armazenar informações específicas. A variável pode ser alterada durante a execução, já a constante permanece imutável.
- **Variáveis globais em C** → São declaradas fora do escopo de qualquer função e podem ser usadas em qualquer parte do código.
- **Variáveis locais em C** → São declaradas dentro do escopo de uma função ou bloco específico e só existem para aquela função.
- **Tipos primitivos** – Servem para especificar o tipo de valor armazenado pelas variáveis:
 - int:** Armazena números inteiros (positivos e negativos).
 - float:** Armazena números com ponto decimal (precisão simples de 32 bits).
 - double:** Armazena números com ponto decimal de maior precisão e capacidade (precisão dupla com 64 bits).
 - Char:** Armazena uma letra ou símbolo usando um único byte, de acordo com a tabela ASCII.
 - Void:** Indica a ausência de um tipo ou valor, comumente usado em funções que não retornam nada.

- **Modificadores de Tipos** – Serve para alterar o tamanho ou o sinal dos tipos numéricos:

- signed:** Permite números com sinal (positivo e negativo)
- unsigned:** Guarda apenas números positivos, dobrando o limite máximo do valor positivo.
- short:** Diminui o espaço de memória para números inteiros
- long:** Aumenta o espaço de memória para os tipos numéricos

Tipo	Num de bits	Intervalo	
		Início	Fim
char	8	-128	127
unsigned char	8	0	255
signed char	8	-128	127
int	16	-32.768	32.767
unsigned int	16	0	65.535
signed int	16	-32.768	32.767
short int	16	-32.768	32.767
unsigned short int	16	0	65.535
signed short int	16	-32.768	32.767
long int	32	-2.147.483.648	2.147.483.647
signed long int	32	-2.147.483.648	2.147.483.647
unsigned long int	32	0	4.294.967.295
float	32	3,4E-38	3,4E+38
double	64	1,7E-308	1,7E+308
long double	80	3,4E-4932	3,4E+4932

- **Operadores lógicos, aritméticos e relacionais** - símbolos especiais usados para fazer contas matemáticas, comparar valores, juntar regras lógicas e alterar dados em variáveis:

Operador	Ação
+	Soma
-	Subtração
*	Multiplicação
/	Divisão
%	Resto de divisão Inteira
++	Incremento
--	Decremento

Tipo	Operador	Sintaxe
Menor que	<	a < b
Maior que	>	a > b
Menor ou igual que	<=	a <= b
Maior ou igual que	>=	a >= b
Igual a	==	a == b
Diferente de	!=	a != b
Não (Lógico)	!	!a
E (Lógico)	&&	a && b
Ou (Lógico)		a b

- **Printf** → Função padrão para saída de dados no terminal. O texto a ser exibido deve estar entre aspas duplas (no primeiro parâmetro) e variáveis a serem exibidas devem vir como parâmetros (indicada no texto pelos formatadores). Exemplo: `printf("Idade: %d\n", idade);`
- **Scanf** → Função padrão para a entrada de dados. Na sintaxe, devem ser especificados o tipo do valor a ser lido e o endereço de memória da variável que vai armazenar o valor. Exemplo: `scanf("%d", &numero);`

- **Formatadores de Tipo** – são códigos iniciados por um sinal de porcentagem usados para indicar o tipo de dado que será exibido ou lido:

código	significado
%d	decimal
%f	ponto flutuante
%c	caracter simples
%s	cadeia de caracter
%e	notação científica
%o	octal
%u	decimal sem sinal
%x	hexadecimal
%ld	decimal longo
%lf	ponto flutuante longo